МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации информационных технологий

Отчет по практическим работам №5-8

по дисциплине «Системная и программная инженерия»

Выполнили:
Студенты группы ИНБО-12-23 Албахтин И.В.
Губарев С.А.
Мосяж М. О.
Симонов Е.Ю.

Проверил: Преподаватель Куликова И.В.

2026 г.

СОДЕРЖАНИЕ

5 СОЗДАНИЕ СТРУКТУРНОЙ ДИАГРАММЫ СИСТЕМЫ 4
5.1 Цель практической 4
5.2 Диаграмма классов системы 4
5.3 Диаграмма объектов 5
5.4 Моделирование процесса работы в нотации BPMN 2.0 7
5.5 Заключение по ПР5 10
6 СОЗДАНИЕ ИНФОРМАЦИОННОЙ ДИАГРАММЫ СИСТЕМЫ И ЛОГИЧЕСКОЙ МОДЕЛИ БАЗЫ ДАННЫХ 11
6.1 Цель практической 11
6.2 Диаграмма потоков данных в нотации DFD 11
6.3 Описание информационного взаимодействия компонентов системы 13
6.4 Логическая модель базы данных в третьей нормальной форме 13
6.5 Заключение по ПР6 15
7 АРХИТЕКТУРА СИСТЕМЫ 16
7.1 Цель практической 16
7.2. Описание архитектуры системы 16
7.3 Обоснование технологического стека 17
7.4 Архитектурная диаграмма разработки 19
7.5 Матрица требований 20
7.5 Заключение по ПР7 22
8 ФОРМИРОВАНИЕ ТЕХНИЧЕСКОГО ЗАДАНИЯ 23
8.1 Цель практической 23
8.2 Обоснование выбора стандарта ГОСТ 34.602-2020 23
8.3 Разработка разделов технического задания на автоматизированную систему 24

8.4. Техническое задание на автоматизированную систему 25
8.4.1 Общие сведения 25
8.4.2 Назначение и цели создания системы..... 25
8.4.3 Характеристика объекта автоматизации 26
8.4.4 Требования к системе 27
8.4.5 Состав и содержание работ по созданию системы..... 30
8.4.6 Порядок разработки автоматизированной системы..... 31
8.4.7 Порядок контроля и приёмки системы..... 31
8.4.8 Требования к документированию 31
8.4.9 Источники разработки..... 32
8.5 Заключение по ПР8..... 32
ЗАКЛЮЧЕНИЕ 34

5 СОЗДАНИЕ СТРУКТУРНОЙ ДИАГРАММЫ СИСТЕМЫ

5.1 Цель практической

Основной целью данной работы является переход от функциональных требований к проектированию внутренней структуры системы «TeaIt» с использованием языка моделирования UML и графических нотаций бизнес-процессов. Для достижения этой цели необходимо разработать диаграммы классов и объектов, которые визуализируют иерархию данных, их атрибуты и взаимосвязи. Также требуется провести детальный анализ и моделирование одного из ключевых процессов приложения (например, добавление записи о напитке) в нотации BPMN 2.0 или IDEF0 для обеспечения однозначного понимания алгоритмов работы системы всеми участниками команды.

5.2 Диаграмма классов системы

Диаграмма классов описывает статическую структуру системы TeaIt – состав сущностей, их атрибуты, методы и связи между ними. В диаграмме выделено восемь классов, каждый из которых отвечает за конкретный аспект работы приложения.

AppSettings хранит пользовательские настройки: дневную норму в миллилитрах, тему оформления, язык и флаг включения уведомлений. DrinkType – простой справочник типов напитков (вода, чай, кофе и т. д.). DrinkEntry представляет одну запись о выпитом напитке: дату и время, объём и ссылку на тип; класс также содержит метод редактирования записи.

ConsumptionHistory агрегирует все записи DrinkEntry и предоставляет методы для добавления, удаления и выборки записей по дате или периоду. DailyProgress рассчитывает прогресс за текущий день: потребляемый объём и процент выполнения нормы – опираясь на историю и настройки. Statistics предоставляет методы для получения агрегированных объёмов за день и за произвольный период, а также построения диаграммы дня. NotificationService следит за прогрессом и отправляет уведомления при достижении 50% и 100% нормы. LocalDatabase обеспечивает персистентность: сохраняет и загружает как записи DrinkEntry, так и настройки AppSettings.

Ключевые связи: DrinkEntry ссылается на DrinkType (каждая запись имеет ровно один тип); ConsumptionHistory агрегирует произвольное количество DrinkEntry; DailyProgress и Statistics зависят от ConsumptionHistory; DailyProgress также зависит от AppSettings для получения целевой нормы; NotificationService зависит от DailyProgress; LocalDatabase хранит как DrinkEntry, так и AppSettings. Диаграмма представлена на рисунке 5.1.

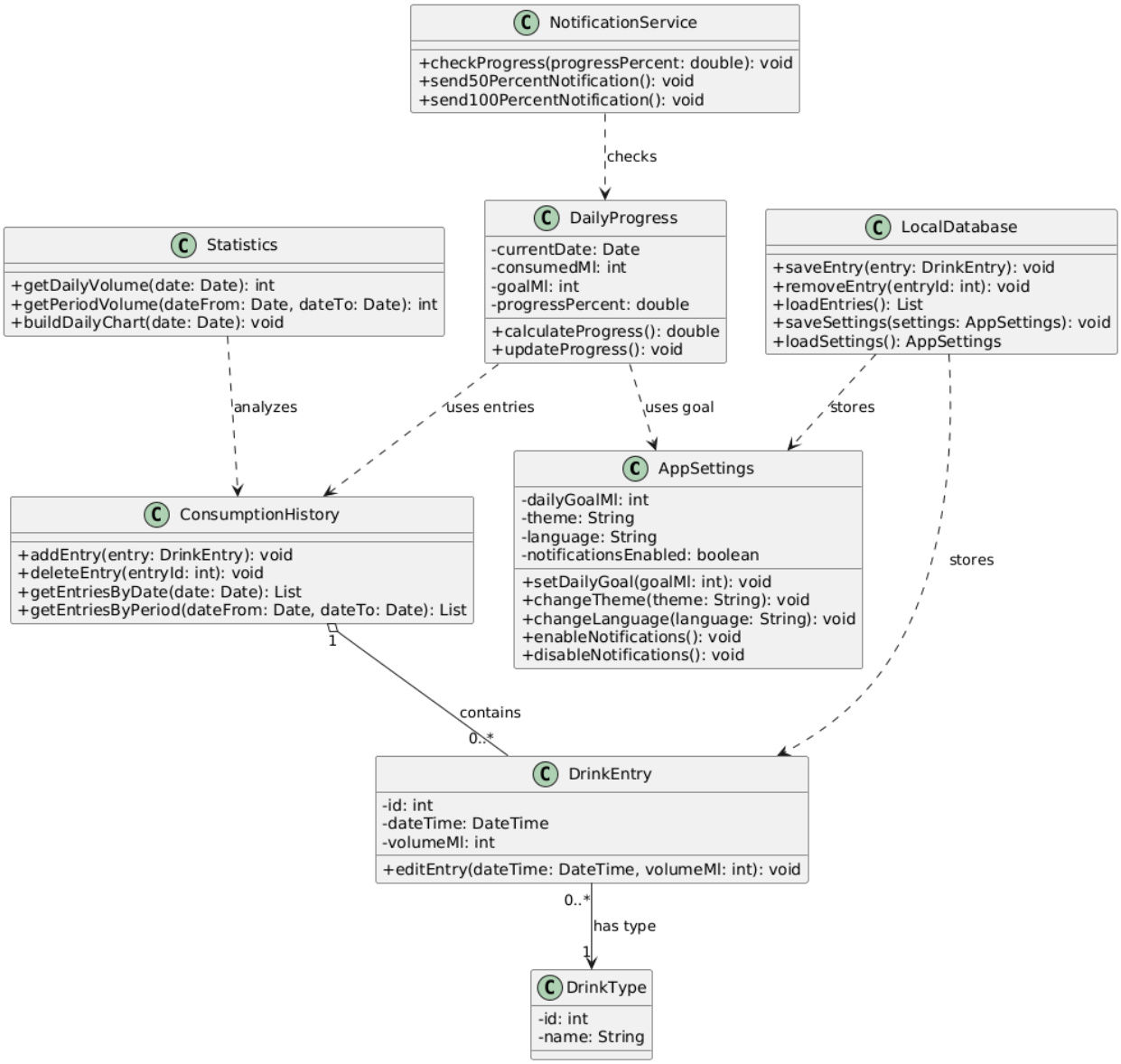

Рисунок 5.1 – Диаграмма классов системы TeaIt

5.3 Диаграмма объектов

Диаграмма объектов фиксирует конкретное состояние системы в момент времени и позволяет убедиться, что структура классов корректно работает на

реальных данных. В качестве примера взят сеанс пользователя 3 апреля 2026 года.

Объект appSettings1 содержит дневную норму 2000 мл, тёмную тему, язык «ru» и включённые уведомления. В справочнике присутствуют два объекта типа напитков: drinkType1 («Чай») и drinkType2 («Вода»). Объект drinkEntry1 (id=101) отражает порцию чая объёмом 300 мл, добавленную в 09:00, и связан с drinkType1. Объект drinkEntry2 (id=102) – 500 мл воды в 13:20, связан с drinkType2.

Объект history1 агрегирует обе записи. На его основе объект dailyProgress1 вычисляет: потреблено 800 мл из 2000 мл цели (взятой из appSettings1), прогресс – 40%. Объекты statistics1 и notificationService1 связаны с history1 и dailyProgress1 соответственно, реализуя агрегацию и контроль уведомлений. Объект localDatabase1 ссылается на appSettings1 и обе записи – отражая, что именно он персистирует. Диаграмма объектов представлена на рисунке 5.2.

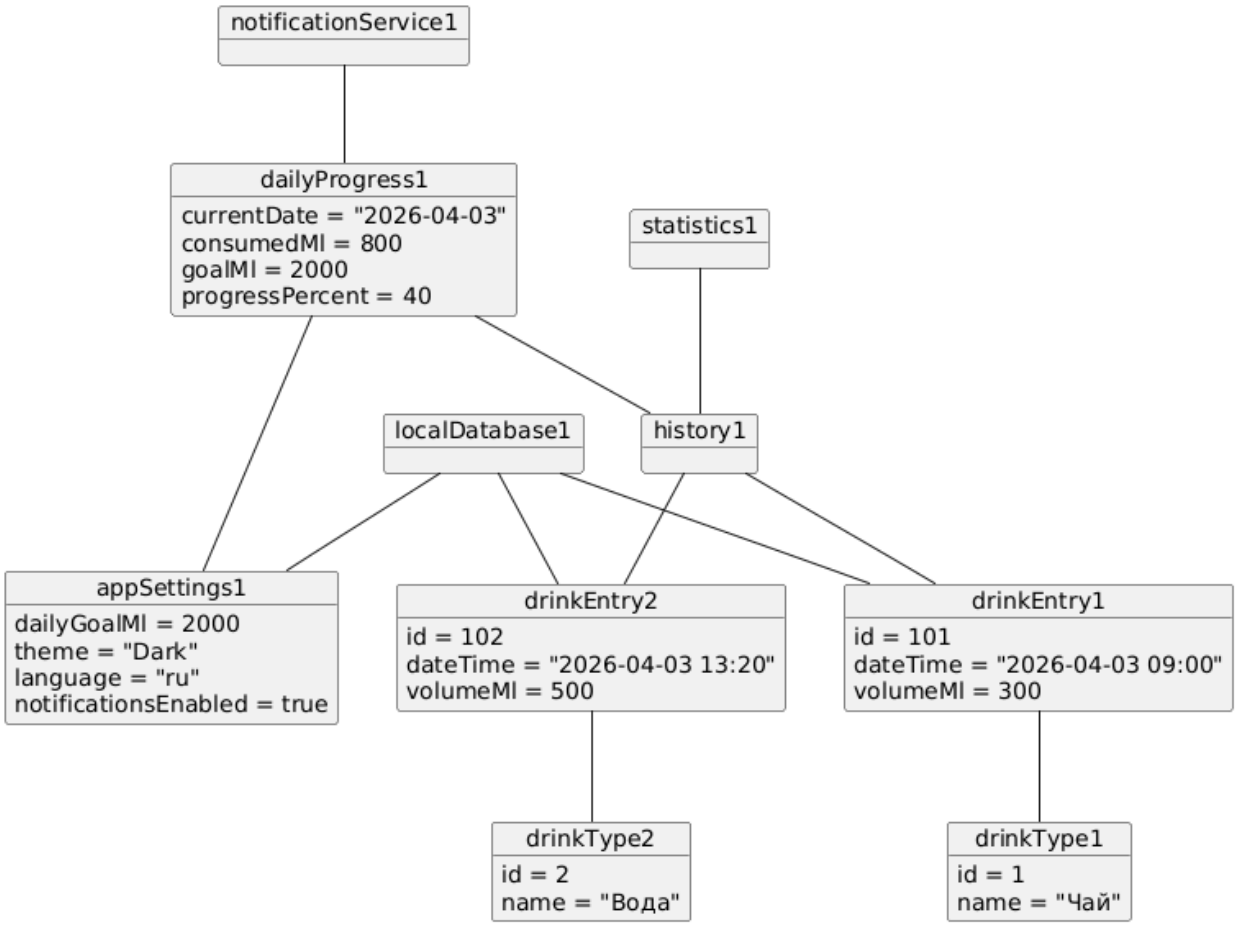

Рисунок 5.2 – Диаграмма объектов системы TeaIt

5.4 Моделирование процесса работы в нотации BPMN 2.0

Для визуализации бизнес-логики приложения построена BPMN 2.0-диаграмма процесса добавления новой записи о выпитом напитке. Процесс разбит на три дорожки (Pools/Lanes): Пользователь, Мобильное приложение и Локальная база данных – что явно разграничивает ответственность каждой стороны.

Процесс стартует событием «Пользователь захотел внести запись». Далее пользователь открывает приложение и нажимает кнопку добавления. Приложение открывает экран ввода. Пользователь вводит данные (тип напитка и объём) и нажимает «Сохранить». После этого приложение выполняет проверку корректности введённых данных: если данные некорректны – пользователь возвращается к экрану ввода (цикл через Exclusive Gateway); если данные корректны – процесс продолжается.

В случае успешной валидации приложение отправляет запрос на

сохранение в базу данных. База данных сохраняет запись и возвращает подтверждение. Приложение обновляет интерфейс (пересчитывает прогресс дня, обновляет список). Процесс завершается. Диаграмма показывает, что ни пользователь, ни база данных не взаимодействуют напрямую – вся координация проходит через слой приложения. Диаграмма BPMN представлена на рисунке 5.3.

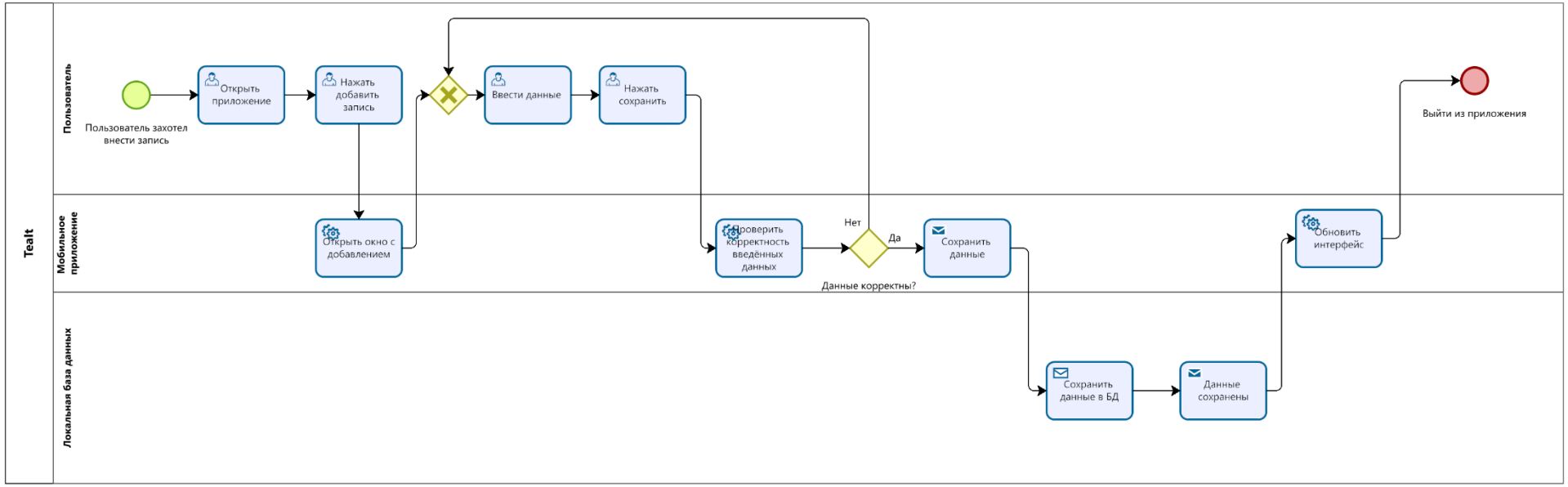

Рисунок 5.3 – BPMN 2.0-диаграмма процесса добавления записи о напитке

5.5 Заключение по ПР5

В ходе выполнения работы была спроектирована статическая структура приложения «TeaIt». Построенная диаграмма классов определила основные сущности системы (Пользователь, Напиток, История, Уведомление) и их методы, а диаграмма объектов позволила протестировать структуру на конкретных примерах данных. Моделирование в нотации BPMN 2.0 обеспечило наглядное представление логики работы приложения, что минимизирует риски разночтений при дальнейшей разработке.

6 СОЗДАНИЕ ИНФОРМАЦИОННОЙ ДИАГРАММЫ СИСТЕМЫ И ЛОГИЧЕСКОЙ МОДЕЛИ БАЗЫ ДАННЫХ

6.1 Цель практической

Целью работы является проектирование информационных потоков и структуры хранения данных для обеспечения корректного функционирования мобильного приложения «TeaIt». В рамках практики необходимо построить диаграмму потоков данных (DFD), описывающую взаимодействие внешних сущностей, процессов и хранилищ, а также разработать нормализованную логическую модель базы данных. Это позволит четко определить, какие данные необходимы для обработки функций системы и как они будут организованы для обеспечения целостности и отсутствия избыточности.

6.2 Диаграмма потоков данных в нотации DFD

Для приложения TeaIt построена двухуровневая DFD в нотации Йордана: контекстная диаграмма (уровень 0) и детализированная диаграмма (уровень 1).

На уровне 0 единственная внешняя сущность – Пользователь – взаимодействует с системой как с целым: передаёт команды и получает результаты. На уровне 1 система раскрывается в четыре процесса: 1.0 – Управление профилем потребления (настройка нормы и напоминаний); 2.0 – Управление напитками (добавление, изменение, удаление записей); 3.0 – Формирование статистики (агрегация и отображение данных); 4.0 – Управление настройками приложения (тема, язык).

Данные хранятся в трёх хранилищах: D1 – база записей о напитках (DrinkEntries); D2 – пользовательские настройки (норма, напоминания, тема, язык); D3 – статистика потребления (предварительно агрегированные данные). Процессы 1.0 и 4.0 работают с D2; процесс 2.0 читает и пишет D1, а также передаёт данные в D3 для пересчёта статистики; процесс 3.0 читает D1 и D3. Диаграмма DFD представлена на рисунке 6.1.

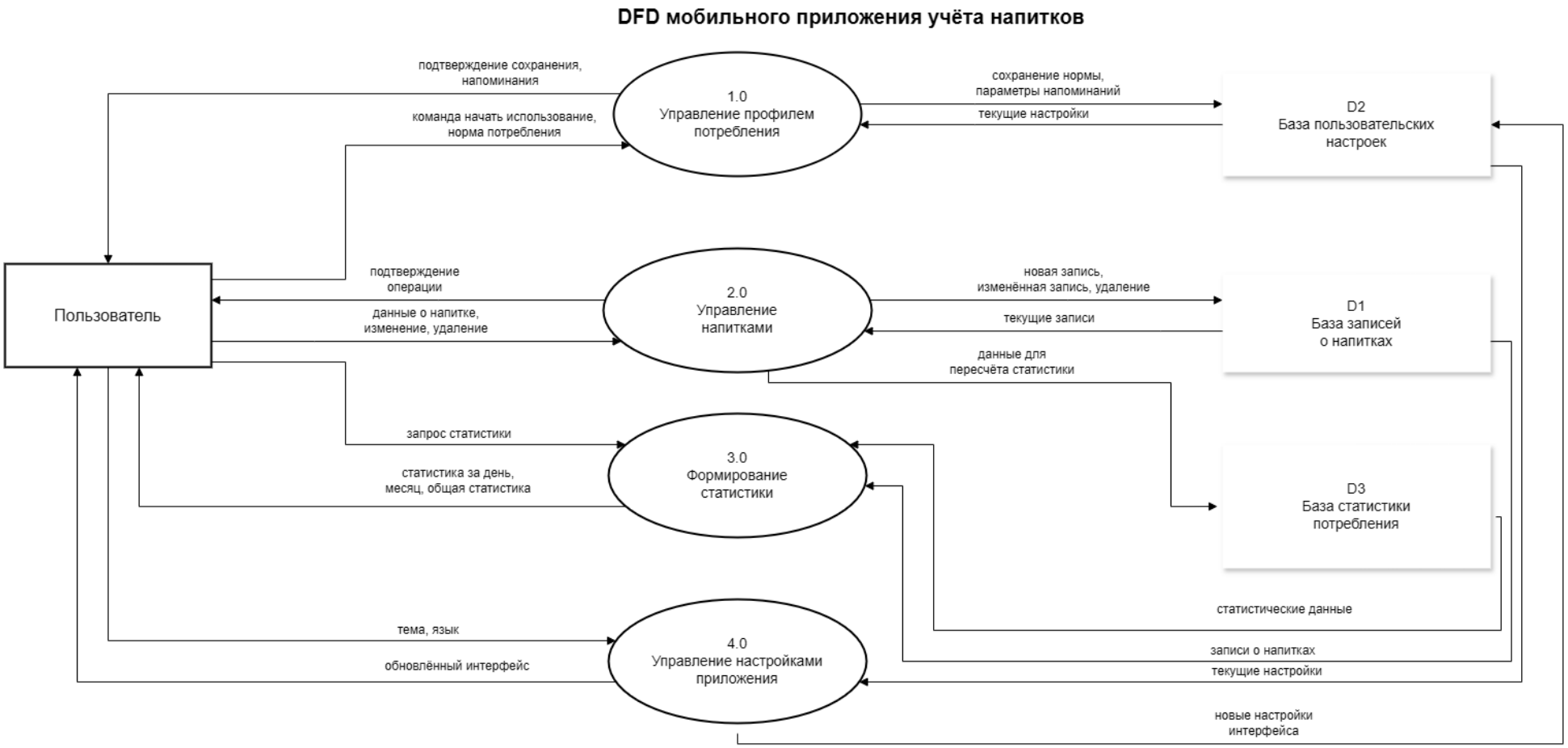

Рисунок 6.1 – Диаграмма потоков данных (DFD) приложения TeaIt

6.3 Описание информационного взаимодействия компонентов системы

Пользователь является единственной внешней сущностью и взаимодействует только с процессами – никогда напрямую с хранилищами. Входящие потоки от пользователя: команды запуска и настройки нормы потребления в процесс 1.0; данные о напитке, запросы на изменение и удаление в процесс 2.0; запрос статистики в процесс 3.0; параметры темы и языка в процесс 4.0. Исходящие потоки к пользователю: подтверждения и напоминания из процесса 1.0; подтверждение операции из процесса 2.0; статистика за день, месяц и общий период из процесса 3.0; обновлённый интерфейс из процесса 4.0.

Процессы не обращаются к хранилищам произвольным образом: каждый работает строго со своей зоной ответственности. Процесс 1.0 читает и записывает норму потребления и параметры напоминаний в хранилище D2. Процесс 2.0 создаёт, изменяет и удаляет записи в D1, а результат каждой операции также передаётся в D3 для немедленного пересчёта агрегированной статистики. Процесс 3.0 является только читающим: он извлекает отдельные записи из D1 и готовые агрегаты из D3, не внося изменений. Процесс 4.0 читает и записывает настройки интерфейса (тема, язык) исключительно в D2, не затрагивая D1 и D3.

Такое разделение обеспечивает независимость процессов: изменение схемы статистики в D3 не затронет логику добавления записей в процессе 2.0, а модификация настроек оформления в D2 не повлияет на расчёты в процессе 1.0. Хранилища D1 и D3 связаны только опосредованно – через процесс 2.0, который служит единственной точкой записи в оба.

6.4 Логическая модель базы данных в третьей нормальной форме

Логическая модель базы данных приложения TeaIt приведена к третьей нормальной форме: нет повторяющихся групп, нет частичных зависимостей от составного ключа, нет транзитивных зависимостей между неключевыми

атрибутами. Модель включает восемь таблиц.

Users (user_id PK, created_at, is_guest) – базовая таблица пользователей, поддерживает работу без регистрации через флаг is_guest. DrinkTypes (drink_type_id PK, name, default_water_ratio) – неизменяемый справочник типов напитков с коэффициентом содержания воды для корректного подсчёта водного баланса. DrinkEntries (entry_id PK, user_id FK, drink_type_id FK, volume_ml, consumed_at, comment) – центральная таблица записей; связана с пользователем и типом напитка.

ConsumptionNorms (norm_id PK, user_id FK, daily_target_ml, start_date, end_date) – хранит историю изменений дневной нормы; end_date = NULL означает актуальную норму. ReminderSettings (reminder_id PK, user_id FK UNIQUE, is_enabled, interval_minutes, start_time, end_time) – настройки напоминаний, один набор на пользователя, что обеспечено ограничением UNIQUE на user_id. Languages (language_id PK, code, name) и Themes (theme_id PK, name) – справочные таблицы, вынесенные отдельно для предотвращения транзитивных зависимостей. UserSettings (setting_id PK, user_id FK UNIQUE, language_id FK, theme_id FK) – персональные настройки интерфейса в отношении 1:1 с пользователем.

Основные связи: Users → DrinkEntries (1:M), Users → ConsumptionNorms (1:M), Users → ReminderSettings (1:1), Users → UserSettings (1:1), DrinkTypes → DrinkEntries (1:M), Languages → UserSettings (1:M), Themes → UserSettings (1:M). Логическая модель представлена на рисунке 6.2.

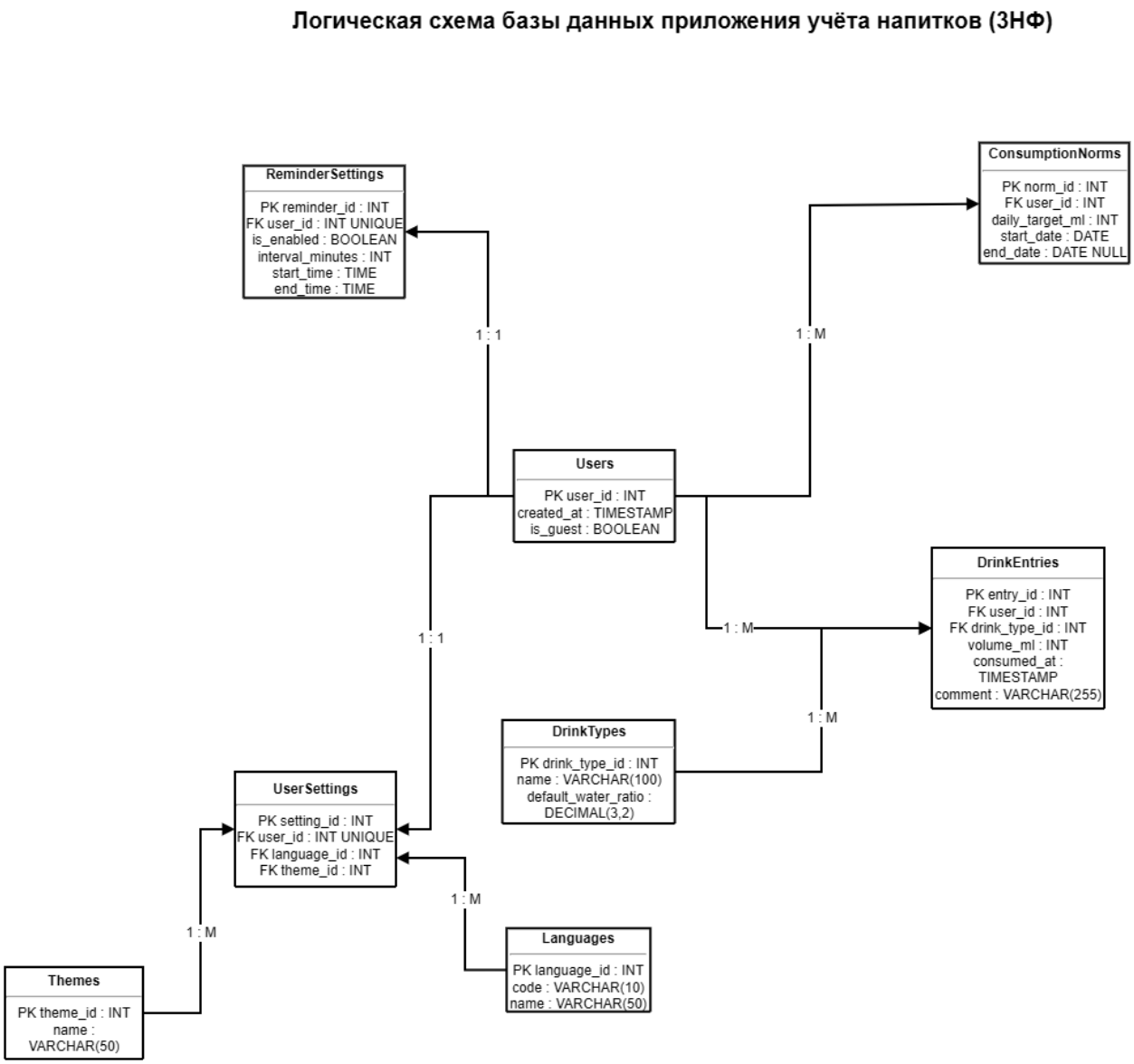

Рисунок 6.2 – Логическая схема базы данных приложения TeaIt (3НФ)

6.5 Заключение по ПР6

По итогам практики была сформирована полная информационная модель системы. Диаграмма DFD позволила визуализировать пути движения данных от пользователя к внутренним модулям обработки и базе данных. Разработанная логическая модель БД, приведенная к необходимым нормальным формам, заложила фундамент для физической реализации хранилища данных, обеспечив эффективное управление информацией о потреблении напитков.

7 АРХИТЕКТУРА СИСТЕМЫ

7.1 Цель практической

Целью работы является определение и обоснование высокоуровневой архитектуры приложения «TeaIt», а также выбор стека технологий для его реализации. Необходимо обосновать выбор языков программирования, фреймворков и СУБД, исходя из критериев эффективности, гибкости и расширяемости системы. Важной задачей является построение архитектурной диаграммы и демонстрация того, как выбранные решения позволяют удовлетворить нефункциональные требования (производительность, безопасность, доступность), сформулированные на этапе ПР4.

7.2. Описание архитектуры системы

TeaIt представляет собой нативное Android-приложение, не имеющее серверной части и работающее полностью на устройстве пользователя. Поэтому для него была выбрана одноуровневая (однозвенная) архитектура, при которой все компоненты системы – пользовательский интерфейс, бизнес-логика и хранилище данных – выполняются в рамках единого процесса на одной аппаратной платформе (Android-смартфон).

Согласно классификации, приведённой в лекционном материале, одноуровневая архитектура означает физическое единство всех слоёв: несмотря на то что внутри приложения выделены слои представления, логики и данных, они не разделены по отдельным серверам или процессам и не взаимодействуют через сеть. Такой подход полностью соответствует характеристикам проекта: приложение ориентировано на офлайн-работу, не требует синхронизации с сервером и рассчитано на индивидуальное использование.

Внутри единственного уровня выделены три логических слоя:

– Слой представления – экраны и элементы пользовательского интерфейса, реализованные средствами Jetpack Compose: MainActivity, HomeScreen, HorizontalPager, AlertDialog, DatePicker, пакет ui/theme.

– Слой бизнес-логики – компоненты, инкапсулирующие правила предметной области: DrinkLogic.kt (операции над записями), DrinkCatalog.kt (справочник напитков), GreetingUtils.kt (вспомогательная логика приветствий), AppModels.kt (модели данных).

– Слой данных – механизм персистентного хранения: TeaItStorage.kt как единая точка доступа, SharedPreferences в качестве физического хранилища, JSON-сериализация для представления объектов.

7.3 Обоснование технологического стека

7.3.1 Язык программирования – Kotlin

Kotlin является официальным рекомендуемым языком разработки для платформы Android начиная с 2019 года. По сравнению с Java он обеспечивает статическую null-безопасность на уровне системы типов, что устраняет целый класс NullPointerException-ошибок ещё на этапе компиляции, а не в рантайме. Kotlin предоставляет расширения (extension functions), data-классы с автоматической генерацией equals/hashCode/copy и деструктуризацией, а также корутины – легковесный механизм асинхронности, встроенный в стандартную библиотеку. Для команды из четырёх человек, работающей в сжатые сроки, преимущество Kotlin состоит в значительном сокращении объёма boilerplate-кода: типичный класс-модель данных на Kotlin занимает одну строку (data class), тогда как эквивалент на Java требует десятков строк с геттерами, сеттерами и конструктором. Это напрямую повышает читаемость кода и снижает вероятность ошибок при командной работе.

7.3.2 Jetpack Compose и Material 3

Jetpack Compose – декларативный UI-фреймворк Google, ставший основным рекомендуемым подходом к разработке пользовательского интерфейса на Android. В отличие от традиционного подхода на основе XML-макетов и View-иерархии, Compose описывает интерфейс как функцию от состояния: при изменении данных фреймворк автоматически перерисовывает только затронутые части UI, что устраняет ручную синхронизацию между данными и отображением. Для приложения TeaIt это принципиально важно:

все экраны реагируют на изменения списка записей, дневной нормы и прогресса в реальном времени без дополнительного кода обновления.

Material 3 (Material You) – актуальная дизайн-система Google, реализующая адаптивную цветовую схему под обои устройства и современные компоненты (NavigationBar, FilledButton, DatePicker нового поколения и др.). Использование Material 3 избавляет команду от создания кастомного дизайна с нуля и гарантирует соответствие ожиданиям пользователей Android-экосистемы.

HorizontalPager применяется для горизонтальной навигации между вкладками (главная, история, статистика) без подключения внешних навигационных библиотек – простое и производительное решение для трёх экранов приложения. AlertDialog и DatePicker используются как готовые компоненты Material 3 для диалогов подтверждения и выбора даты, что исключает необходимость реализации этих элементов вручную.

7.3.3 Single-Activity (ComponentActivity)

Паттерн Single-Activity означает, что всё приложение работает в рамках одной Activity (MainActivity), а навигация между экранами реализована внутри Compose-иерархии, а не через отдельные Activity или Fragment. Это стандартный и рекомендованный архитектурный паттерн при использовании Jetpack Compose: он устраняет сложности управления жизненным циклом нескольких Activity, упрощает передачу состояния между экранами (состояние хранится на уровне Activity или ViewModel) и снижает накладные расходы на переключение между экранами. ComponentActivity – базовый класс из библиотеки androidx.activity, предоставляющий поддержку всех современных Jetpack-компонентов без лишней функциональности устаревшего AppCompatActivity.

7.3.4 Локальное хранение

Выбор SharedPreferences как механизма хранения данных вместо полноценной базы данных (Room/SQLite) обоснован характеристиками предметной области: объём данных невелик (записи о напитках за несколько

месяцев не превышают нескольких тысяч объектов), структура данных проста и плоска (список однородных объектов DrinkEntry без связей типа «один-ко-многим»), отсутствуют сложные запросы на фильтрацию или группировку, требующие SQL. SharedPreferences встроены в Android SDK, не требуют дополнительных зависимостей в Gradle и не усложняют сборку проекта. JSON-сериализация позволяет хранить весь список записей как одну строку по ключу, что достаточно для данного масштаба и позволяет легко мигрировать схему при добавлении новых полей.

Компонент TeaItStorage.kt выступает единственной точкой доступа к данным, инкапсулируя все операции чтения и записи. Это соответствует принципу единственной ответственности (Single Responsibility) и позволяет в будущем заменить механизм хранения (например, на Room) без изменения логики выше по стеку.

7.3.5 Система сборки

Gradle – стандартная система сборки для Android-проектов, обязательная при работе с Android SDK. Задача :app:assembleDebug генерирует отладочную APK с дефолтным debug-ключом подписи, что достаточно для установки на устройство разработчика и проведения тестирования. Использование debug-варианта сборки на этапе разработки освобождает команду от необходимости управлять production-ключами и настраивать ProGuard/R8, что ускоряет итерационный цикл разработки.

7.4 Архитектурная диаграмма разработки

Архитектурная диаграмма приложения TeaIt представлена ниже на рисунке 7.1. Поскольку все компоненты выполняются на одном устройстве, диаграмма отражает одноуровневую архитектуру с тремя логическими слоями внутри единственного уровня.

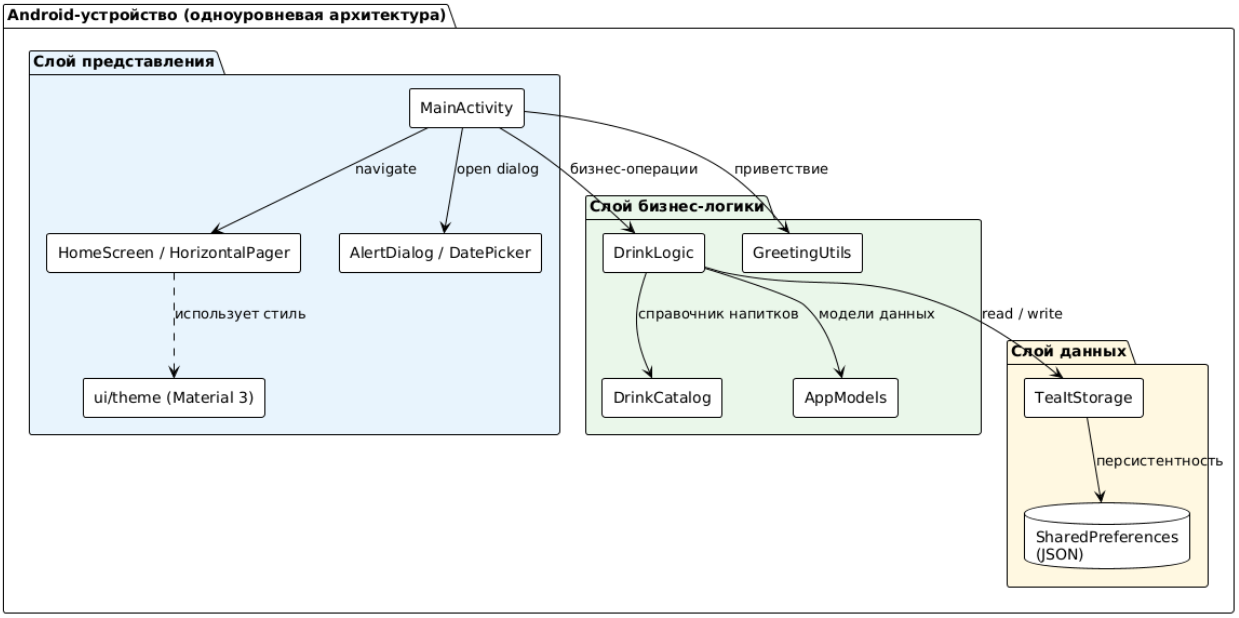

Рисунок 7.1 – Одноуровневая архитектура приложения TealT

Из диаграммы видно, что поток управления однонаправлен: слой представления обращается к слою бизнес-логики, который, в свою очередь, взаимодействует со слоем данных. Обратного прямого доступа UI к хранилищу нет – это обеспечивает изоляцию логики и единственную ответственность каждого компонента. Все три слоя работают в рамках единого Android-процесса на устройстве пользователя, что подтверждает одноуровневый характер архитектуры.

7.5 Матрица требований

В таблице 7.1 приведена матрица требований, дополненная столбцом «Компоненты архитектуры», в котором для каждого требования указаны конкретные компоненты системы, обеспечивающие его реализацию.

Таблица 7.1 – Дополненная матрица требований

№	Требование	Суть	Автор	Ссылки	Критерий проверки	Компоненты архитектуры
1.1	Запуск без регистрации	Приложение должно обеспечивать возможность начала использования без регистрации, авторизации и прохождения онбординга.	Команда	healthline.com	Приложение готово к работе не более чем за 3 сек после запуска	MainActivity (слой представления); TeaItStorage (слой данных – проверка первого запуска)
1.2	Задание дневной нормы	Приложение должно обеспечивать возможность установки пользователем индивидуальной дневной нормы потребления жидкости.	Команда	efsa.onlinelibrary.wiley.com	Прогресс пересчитывается автоматически	AlertDialog (слой представления); DrinkLogic (слой логики); TeaItStorage/SharedPreferences (слой данных)
1.3	Добавление записи на главной странице	Приложение должно обеспечивать удобное и быстрое добавление новых записей о потреблении жидкости.	Команда	healthline.com	Добавление сразу нескольких записей без дополнительных действий	HomeScreen / AlertDialog / DatePicker (слой представления); DrinkLogic, DrinkCatalog, AppModels (слой логики); TeaItStorage (слой данных)
1.4	Редактирование записи	Приложение должно обеспечивать возможность редактирования ранее сохранённых записей о потреблении жидкости.	Команда	US-04	Обновлённые данные видны сразу, не позднее чем через 1 сек после сохранения	AlertDialog / DatePicker (слой представления); DrinkLogic, AppModels (слой логики); TeaItStorage (слой данных)
1.5	Удаление записи	Приложение должно обеспечивать возможность удаления записей с обязательным подтверждением действия.	Команда	nngroup.com	Запись удаляется только после подтверждения	AlertDialog (слой представления); DrinkLogic (слой логики); TeaItStorage (слой данных)
1.6	Прогресс дня	Приложение должно отображать текущий прогресс потребления жидкости за день относительно установленной дневной нормы.	Команда	healthline.com	Прогресс отображается с точностью ±5 мл после каждой операции	HomeScreen (слой представления); DrinkLogic (слой логики)
1.7	Визуализация паттернов потребления	Приложение должно обеспечивать графическое отображение потребления жидкости: круговую диаграмму для текущего дня и мини-круговые диаграммы для каждого дня месяца.	Команда	docs.dhis2.org	Диаграмма дня обновляется после каждой записи	HomeScreen / HorizontalPager (слой представления); DrinkLogic (слой логики)

№	Требование	Суть	Автор	Ссылки	Критерий проверки	Компоненты архитектуры
1.8	Агрегированная статистика	Приложение должно отображать суммарный объём потребления жидкости за выбранный пользователем период времени.	Команда	watermin der.com	Суммы совпадают с расчётами с точностью ±1%	HorizontalPager / StatisticsScreen (слой представления); DrinkLogic (слой логики); TeaItStorage / SharedPreferences (слой данных)
1.9	Офлайн-режим	Приложение должно обеспечивать полноценную работу всех функций без подключения к интернету, с локальным хранением данных.	Команда	US-09	Работа в авиарежиме и без интернета	TeaItStorage / SharedPreferences (слой данных – вся персистентность локальная)
1.10	Уведомления	Приложение должно обеспечивать отправку уведомлений пользователю в течение дня о достижении определённого уровня прогресса.	Команда	watermin der.com	Уведомления приходят при 50% и 100%	Android SDK NotificationManager (слой представления / системный); DrinkLogic (слой логики)

7.5 Заключение по ПР7

В рамках практической работы была определена архитектура приложения, обеспечивающая баланс между скоростью разработки и качеством продукта. Выбранный технологический стек был обоснован требованиями к нативности, надёжности, производительности на устройствах с API 26и, а также ограниченным объёмом хранимых данных. Построенная архитектурная диаграмма подтверждает, что система способна масштабироваться и выдерживать заданные нагрузки, при этом оставаясь простой в сопровождении и тестировании.

8 ФОРМИРОВАНИЕ ТЕХНИЧЕСКОГО ЗАДАНИЯ

8.1 Цель практической

Основной целью завершающей работы является консолидация всех результатов проектирования в единый юридически и технически значимый документ – Техническое задание (ТЗ). Для этого необходимо обосновать выбор государственного стандарта и составить разделы ТЗ, включающие общие сведения, назначение, цели создания и подробные требования к системе. Работа направлена на формирование исчерпывающего описания продукта, которое послужит эталоном при разработке, контроле и приемке готовой системы.

8.2 Обоснование выбора стандарта ГОСТ 34.602-2020

Задание предлагает выбор между двумя стандартами: ГОСТ 34.602-2020 – на техническое задание для автоматизированных систем, и ГОСТ 19.201-78 – на техническое задание для программ и программных изделий.

ГОСТ 19.201-78 разработан для документирования программного кода: он фокусируется на функциональных характеристиках, условиях эксплуатации и программной документации. Его структура подходит для описания автономного программного модуля, но не предусматривает описания архитектуры, компонентов хранения, информационного обеспечения и требований к надёжности системы в целом. Для TeaIt, у которого выделены отдельные архитектурные слои, логическая модель БД и матрица нефункциональных требований, этого недостаточно.

ГОСТ 34.602-2020 ориентирован на автоматизированные системы и содержит разделы, которые напрямую покрывают результаты всех предыдущих практических работ. Раздел «Характеристика объекта автоматизации» соответствует описанию предметной области и портрету пользователя (ПР3). Раздел «Требования к системе» с подразделами по функциям, надёжности, видам обеспечения охватывает матрицу функциональных и нефункциональных требований (ПР2, ПР4). Раздел по информационному обеспечению отражает логическую модель БД (ПР6).

Раздел по программному обеспечению содержит обоснование стека технологий (ПР7). Кроме того, разделы «Состав и содержание работ» и «Порядок контроля и приёмки» позволяют формализовать план разработки и критерии приёмки, что придаёт документу практическую ценность.

Таким образом, ГОСТ 34.602-2020 выбран как стандарт, структура разделов которого максимально соответствует уже сформированным проектным артефактам и позволяет консолидировать их в единый нормативный документ без искусственного переформатирования.

8.3 Разработка разделов технического задания на автоматизированную систему

Анализ задания. ГОСТ 34.602-2020 содержит десять обязательных разделов. Применительно к проекту TeaIt ряд разделов заполняется напрямую из уже готовых материалов, а часть требует дополнительной проработки.

Разделы, заполняемые на основе готовых артефактов: «Общие сведения» – из состава команды и целей дисциплины; «Назначение и цели создания системы» – из описания предметной области (ПР2); «Характеристика объекта автоматизации» – из портрета пользователя и сценариев использования (ПР3); «Требования к системе» (функциональные и нефункциональные) – из матрицы требований (ПР4); требования к информационному обеспечению – из логической модели БД (ПР6); требования к программному обеспечению – из обоснования стека технологий (ПР7).

Разделы, требующие дополнительного формирования: «Состав и содержание работ по созданию системы» – необходимо составить перечень этапов разработки с ориентировочными сроками; «Порядок контроля и приёмки» – определить виды испытаний и критерии приёмки на основе критериев проверки из матрицы требований; «Требования к документированию» – перечислить состав проектной документации; «Источники разработки» – оформить список ссылок, использованных при разработке требований.

Разделы, неприменимые к данному проекту и исключаемые согласно п. 4.3 ГОСТ 34.602-2020: «Требования к транспортабельности» (подвижные АС), «Требования к метрологическому обеспечению» (измерительные каналы), «Требования к патентной чистоте» (в рамках учебного проекта не актуально). Стандарт прямо допускает исключение неприменимых подразделов.

8.4. Техническое задание на автоматизированную систему

8.4.1 Общие сведения

Полное наименование системы: мобильное приложение для учёта потребления жидкости «TeaIt». Условное обозначение: TeaIt-AS.

Разработчик: студенческая команда группы ИНБО-12-23 РТУ МИРЭА (Албахтин И.В., Губарев С.А., Мосяж М.О., Симонов Е.Ю.). Заказчик: кафедра математического обеспечения и стандартизации информационных технологий РТУ МИРЭА.

Основание для разработки: задание на практические работы по дисциплине «Системная и программная инженерия», утверждённое кафедрой в 2026 г.

Плановые сроки: начало – февраль 2026 г., завершение – апрель 2026 г. Финансирование не предусмотрено; работа выполняется в рамках учебного процесса.

Результаты работ предъявляются заказчику в виде отчётной документации (отчёты по ПР1–4 и ПР5–8) и настоящего Технического задания.

8.4.2 Назначение и цели создания системы

8.4.2.1 Назначение системы

Система предназначена для автоматизации самостоятельного учёта потреблённых напитков в повседневных условиях. Объект автоматизации – деятельность пользователя по ведению личного журнала потребления жидкости. Система функционирует на персональном мобильном устройстве пользователя под управлением ОС Android (API 26 и выше) без доступа к

интернету и без обязательной регистрации.

8.4.2.2 Цели создания системы

Цель 1: сократить время фиксации факта потребления напитка до 3 нажатий от открытия приложения. Критерий достижения – добавление записи выполняется не более чем за 3 последовательных действия пользователя.

Цель 2: обеспечить пользователю наглядный контроль дневного прогресса относительно установленной нормы. Критерий – актуальный прогресс отображается на главном экране с погрешностью не более ๓ мл сразу после каждой операции.

Цель 3: предоставить агрегированную статистику за день, неделю и месяц. Критерий – суммарные объёмы совпадают с ручным расчётом с точностью не хуже 1%.

Цель 4: обеспечить полную автономность работы. Критерий – все функции системы работают без подключения к сети (режим авиаперелёта).

8.4.3 Характеристика объекта автоматизации

Объект автоматизации – процесс самостоятельного ведения дневника потребления жидкости. До автоматизации пользователь либо ведёт записи вручную (блокнот, таблицы), либо не ведёт их вовсе. Это приводит к потере данных, ошибкам в расчёте нормы и отсутствию обратной связи в течение дня.

Типичный пользователь системы – человек 18–35 лет, следящий за здоровьем и привыкший к мобильным приложениям. Он использует систему в течение всего дня: вносит записи после каждого употребления напитка, периодически проверяет прогресс. Ключевые сценарии: добавление записи сразу после питья, просмотр итогов дня вечером, настройка нормы при изменении рациона.

Условия эксплуатации: личное Android-устройство, нет требований к постоянному подключению к сети, использование в любом месте и в любое время. Требования к окружающей среде: стандартные условия эксплуатации мобильного устройства согласно документации производителя.

8.4.4 Требования к системе

8.4.4.1 Требования к системе в целом

Структура и функционирование. Система реализована как однослойное (монолитное) мобильное приложение. Внутренняя архитектура разделена на три логических слоя: представление (Jetpack Compose, экраны), бизнес-логика (ViewModel, use cases), данные (Room/SQLite, SharedPreferences). Связь между слоями – через репозитории; внешних интерфейсов (API, облако) нет.

Режимы функционирования: основной – интерактивная работа пользователя в реальном времени; фоновый – отправка уведомлений при 50% и 100% дневной нормы через Android WorkManager. Переключение режимов выполняется системой Android автоматически.

Требования к персоналу. Специального персонала не предусмотрено. Пользователь – один человек, не требующий специальной подготовки; достаточно навыков работы с мобильным устройством. Системный администратор не требуется.

Требования к надёжности. Вероятность безотказной работы при корректных входных данных – не ниже 99%. Потеря данных при штатном завершении приложения не допускается. При аварийном завершении (принудительная остановка ОС) допускается потеря не более одной незафиксированной операции. Время восстановления после аварийного завершения – не более 5 секунд при следующем запуске.

Требования к защите информации. Все пользовательские данные хранятся исключительно локально на устройстве в базе данных Room (SQLite). Передача данных на внешние серверы не предусмотрена. Приложение не запрашивает разрешения на доступ к сети, контактам или геолокации. Защита от несанкционированного доступа обеспечивается стандартными механизмами изоляции приложений Android.

Требования к эргономике. Навигация к любой основной функции должна занимать не более 2 нажатий от главного экрана. Интерфейс поддерживает светлую и тёмную темы. Минимальный размер интерактивного

элемента – 48×48 dp согласно рекомендациям Material Design 3. Язык интерфейса: русский.

Сохранность информации при авариях. При отключении питания в момент записи транзакция в SQLite считается незавершённой и откатывается автоматически – данные не повреждаются. Пользователь должен повторить операцию после перезапуска.

8.4.4.2 Требования к функциям системы

Функция 1.1 – запуск без регистрации. Система должна предоставлять полный функционал без создания учётной записи. При первом запуске создаётся анонимный локальный профиль. Критерий проверки: при первом запуске главный экран отображается не позднее чем через 3 секунды без каких-либо форм регистрации.

Функция 1.2 – установка дневной нормы потребления. Система должна позволять пользователю задать индивидуальную дневную норму потребления жидкости в мл. Значение используется для расчёта прогресса и отправки уведомлений. Критерий: изменение нормы немедленно отражается в показателе прогресса на главном экране.

Функция 1.3 – добавление записи о напитке. Система должна обеспечивать быстрое добавление записи с указанием типа напитка и объёма в мл. Поле времени заполняется текущим моментом автоматически с возможностью изменения. После сохранения запись отображается в истории, прогресс пересчитывается. Критерий: добавление записи выполняется за не более 3 действий.

Функция 1.4 – редактирование записи. Система должна позволять изменять тип напитка, объём и время существующей записи. Изменения сохраняются в базе данных и сразу отражаются в истории и статистике. Критерий: обновлённые данные видны не позднее 1 секунды после подтверждения.

Функция 1.5 – удаление записи. Система должна удалять запись только после явного подтверждения пользователя (диалог подтверждения). После

удаления история и статистика обновляются автоматически. Критерий: запись удаляется только после нажатия кнопки подтверждения в диалоге.

Функция 1.6 – отображение прогресса дня. На главном экране постоянно отображается текущий объём потребления за сутки и процент выполнения нормы. Обновление выполняется после каждой операции без перезапуска экрана. Критерий: прогресс отображается с точностью ๓ мл.

Функция 1.7 – визуализация паттернов потребления. Система отображает круговую диаграмму распределения напитков по типам за текущий день и мини-круговые диаграммы для каждого дня текущего месяца. Критерий: диаграмма дня обновляется после каждой добавленной или удалённой записи.

Функция 1.8 – агрегированная статистика за период. Система предоставляет суммарный объём потребления за выбранный период (день, неделя, месяц). Данные агрегируются на основе записей в базе данных. Критерий: суммарные объёмы совпадают с ручным расчётом с точностью не хуже 1%.

Функция 1.9 – офлайн-режим. Все функции системы работают без подключения к сети. Данные сохраняются исключительно на устройстве. Критерий: полная функциональность в режиме авиаперелёта.

Функция 1.10 – уведомления о прогрессе. Система отправляет push-уведомления при достижении 50% и 100% дневной нормы. Уведомления отправляются только в период времени, настроенный пользователем. Критерий: уведомление приходит в течение 5 минут после достижения порогового значения при условии, что приложение имеет разрешение на уведомления.

8.4.4.3 Требования к видам обеспечения

Информационное обеспечение. База данных реализована на SQLite через библиотеку Room в третьей нормальной форме. Структура включает 8 таблиц: Users, DrinkTypes, DrinkEntries, ConsumptionNorms, ReminderSettings, Languages, Themes, UserSettings. Справочник типов напитков (DrinkTypes)

предзаполняется при первом запуске: вода, чай, кофе, сок, молоко. Пользователь не может удалить предустановленные типы. Входные данные: тип напитка (справочник), объём в мл (целое число, 1–5000), дата и время (по умолчанию – текущие). Все строковые поля ограничены 200 символами.

Программное обеспечение. Операционная система: Android API 26 (Android 8.0) и выше. Язык разработки: Kotlin. UI-фреймворк: Jetpack Compose. Хранение данных: Room 2.x (SQLite), SharedPreferences. Асинхронность: Kotlin Coroutines и Flow. Внедрение зависимостей: Hilt. Уведомления: Android WorkManager. Сторонние библиотеки с сетевым доступом не используются.

Техническое обеспечение. Минимальные требования к устройству: Android-смартфон с API 26и, оперативная память ≥ 2 ГБ, свободное место на накопителе ≥ 50 МБ. Требования к сети отсутствуют – приложение полностью автономно. Специального оборудования не требуется.

8.4.5 Состав и содержание работ по созданию системы

Этап 1 (февраль 2026): обследование предметной области, формирование состава команды, определение концепции, разработка пользовательских историй и функциональных требований, составление матрицы нефункциональных требований.

Этап 2 (февраль–март 2026): проектирование структурных диаграмм (UML: классы, объекты), моделирование бизнес-процессов (BPMN 2.0), разработка диаграммы потоков данных (DFD), проектирование логической модели базы данных в 3НФ.

Этап 3 (март 2026): определение архитектуры системы, обоснование технологического стека, построение архитектурной диаграммы, уточнение матрицы требований с привязкой к компонентам.

Этап 4 (март–апрель 2026): разработка Технического задания по ГОСТ 34.602-2020, консолидация всех проектных артефактов.

Этап 5 (апрель 2026 и далее): реализация приложения в соответствии с ТЗ, модульное и интеграционное тестирование, устранение замечаний,

предъявление заказчику.

8.4.6 Порядок разработки автоматизированной системы

Разработка ведётся итерационно в соответствии с задачами, сформированными в трекере Todoist. Каждая итерация охватывает один функциональный блок (добавление записи, статистика, уведомления и т.д.) и включает: постановку задачи, реализацию, модульное тестирование, ревью кода участниками команды.

Распределение обязанностей соответствует ролям, определённым в ПР1: Мосяж М.О. – управление проектом и разработка; Губарев С.А. – разработка и дизайн; Албахтин И.В. – анализ и тестирование; Симонов Е.Ю. – технический писатель и анализ.

8.4.7 Порядок контроля и приёмки системы

Виды испытаний:

– функциональное тестирование: проверка каждой из 10 функций по критериям, указанным в п. 8.4.4.2; выполняется тестировщиком на реальном Android-устройстве;

– регрессионное тестирование: повторная проверка всех функций после исправления дефектов;

– тестирование офлайн-режима: все операции выполняются при включённом авиарежиме.

Критерии приёмки: все функциональные критерии из п. 8.4.4.2 выполнены; количество открытых критических дефектов – 0; количество открытых некритических дефектов – не более 3, каждый задокументирован. Приёмка оформляется преподавателем в виде отметки в ведомости по итогам защиты.

8.4.8 Требования к документированию

В состав документации входят:

– Отчёт по практическим работам №1–4 (обследование, требования, моделирование, матрица требований);

– Отчёт по практическим работам №5–8 (структурные диаграммы, информационная модель, архитектура, ТЗ) – настоящий документ;

– Техническое задание на АС «TeaIt» (раздел 8.4 настоящего отчёта), оформленное по ГОСТ 34.602-2020.

Документация хранится в общем репозитории команды и предоставляется заказчику в электронном виде (форматы .docx, .pdf).

8.4.9 Источники разработки

1. ГОСТ 34.602-2020. Техническое задание на создание автоматизированной системы. М.: Стандартинформ, 2020.

2. ГОСТ 19.201-78. Техническое задание. Требования к содержанию и оформлению. М.: Стандартинформ, 1978.

3. Healthline. How Much Water Should You Drink Per Day? – healthline.com (источник требований к норме потребления жидкости, US-02).

4. Nielsen Norman Group. Confirmation dialogs: when to use them. – nngroup.com (основание для требования подтверждения при удалении, US-05).

5. WaterMinder – мобильное приложение-аналог для учёта потребления воды (waterminder.com). Использован как референс при формировании требований к уведомлениям и статистике (US-10, 1.8).

6. DHIS2 Documentation. Data Visualization Guide. – docs.dhis2.org (принципы визуализации паттернов потребления, US-07).

7. Android Developers. Jetpack Compose. – developer.android.com/jetpack/compose (обоснование выбора UI-фреймворка, ПР7).

8.5 Заключение по ПР8

В ходе выполнения работы было разработано полноценное Техническое задание на мобильное приложение «TeaIt» в соответствии с ГОСТ 34.602-2020. Документ объединил в себе результаты всех предыдущих этапов проектирования: от пользовательских историй до архитектурных решений. Наличие утвержденного ТЗ обеспечивает прозрачность процесса разработки и устанавливает четкие критерии оценки качества итогового программного

продукта.

ЗАКЛЮЧЕНИЕ

В рамках практических работ №5–8 по дисциплине «Системная и программная инженерия» командой была успешно завершена фаза проектирования и документирования мобильного приложения «TeaIt».

В пятой практической работе выполнено структурное моделирование с использованием UML и BPMN 2.0. В шестой практической спроектированы информационные потоки в нотации DFD и нормализованная логическая модель базы данных. В седьмой практической работе была определена архитектура системы, обоснован выбор программных решений и подтверждено выполнение нефункциональных требований. В восьмой практической работе все проектные решения были формализованы в итоговом Техническом задании, составленном по стандарту ГОСТ 34.602-2020, что завершило этап подготовки к реализации системы.